

Deep Residual Learning for Image Recognition — team3 논문리뷰

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft Research)
arXiv:1512.03385 · ILSVRC 2015 classification task 1위

0. 요약

항목	내용
해결하려는 문제	네트워크가 깊어질수록 오히려 학습 오차조차 올라가는 degradation(성능 저하) 문제
핵심 아이디어	레이어가 목표 함수 $H(x)$ 를 직접 배우는 대신, 잔차 $F(x) = H(x) - x$ 를 배우게 하고 출력은 $F(x) + x$ 로 구성
구현 수단	shortcut connection(스킵 연결) — 추가 파라미터·연산량 거의 없음
검증 데이터셋	ImageNet(ILSVRC 2012), CIFAR-10, PASCAL VOC, MS COCO
대표 성과	152층 ResNet, ImageNet top-5 에러 4.49% (단일 모델, dense+multi-scale) / 3.57% (앙상블, test set) → ILSVRC 2015 classification 1위
추가 성과	ImageNet detection·localization, COCO detection·segmentation 4개 트랙 전부 1위
극한 실험	CIFAR-10에서 1202층까지도 "학습이 된다"는 것을 증명(다만 테스트 성능은 110층보다 낮음)
논문의 성격	왜 되는지에 대한 완결된 이론이라기보다, 경쟁 가설을 하나씩 실험으로 소거하며 도달한 강력한 경험적(empirical) 논문

1. 이 논문이 중요한 이유

2015년 이전까지 "네트워크를 깊게 쌓으면 성능이 좋아진다"는 직관은 있었지만, 실제로는 20층 정도만 넘어가도 학습 자체가 잘 안 되는 벽에 부딪혔다. 이 논문은 그 벽을 shortcut connection이라는 아주 단순한 구조 변경 하나로 돌파했고, 그 결과 100층·150층을 넘어 1000층이 넘는 네트워크까지 학습 가능하다는 것을 보여줬다.

다만 이 논문의 진짜 힘은 "잔차를 배우다"는 아이디어 자체보다, 그 아이디어에 도달하기까지 경쟁 가설을 하나씩 실험으로 지워 나간 방법론과, 구조적 선택 하나(파라미터 없는 identity shortcut)가 연산 경제성·학습 안정성·이후 생태계 전체를 어떻게 바꿔 놨는지에 있다고 볼 수 있다. 이 지름길 구조는 이후 이미지 분류를 넘어 물체 탐지, 분할, 나아가 Transformer 계열 아키텍처(residual stream)에 이르기까지 매우 널리 쓰이는 기본 설계 요소가 됐다.

2. 문제 정의: Degradation Problem

2-1. 이미 해결된 문제 — Vanishing/Exploding Gradient

깊은 네트워크의 고전적 장애물은 기울기 소실/폭발(vanishing/exploding gradient) 문제다. 하지만 이 문제는 논문 시점에 정규화된 초기화(normalized initialization)와 batch normalization 같은 중간 정규화 레이어로 상당 부분 해소되어, 수십 층짜리 네트워크도 SGD로 수렴을 "시작하는 것" 자체는 가능해졌다.

2-2. 진짜 문제 — Degradation (세 가지 용의자를 소거하는 과정)

네트워크가 수렴하기 시작해도 새로운 문제가 드러난다. 깊이가 늘어날수록 정확도가 포화되다가 급격히 나빠지는 degradation 문제다. 깊은 네트워크의 학습 오차가 얇은 네트워크보다 높아지는 이 현상에는 그럴듯한 설명이 최소 세 가지 있었는데, 저자들은 이 셋을 실험으로 하나씩 기각해나간다.

 **Figure 1:** CIFAR-10에서 20층 vs 56층 plain(shortcut 없는) 네트워크를 비교하면, 56층이 training error·test error 모두 20층보다 높다. 더 깊은 모델이 표현력은 더 크므로(20층 구조를 부분집합으로 포함) 이론상 있을 수 없는 역전 현상이다.

용의자	기각 근거
① Overfitting	기각. Overfitting이면 test error만 높아야 하는데, training error 자체 가 더 높다(Figure 1, ImageNet의 Table 2: plain-18 27.94% vs plain-34 28.54%).
② Gradient vanishing/exploding	기각. BN을 쓴 plain net도 순전파 신호의 분산이 0이 아님을 확인했고, 역전파 gradient의 norm도 정상 범위였다. He 초기화 + BN 조합이 이미 이 문제를 상당 부분 해소해줬기 때문에, 애초에 이 시점에서는 용의선상 밖이다.
③ 학습 반복이 부족해서	기각. 반복 횟수를 3배로 늘려도 degradation은 그대로였다.

세 용의자가 전부 기각되면 남는 결론은 하나다 — **표현력의 문제가 아니라 최적화 가능성(optimizability)의 문제**라는 것. 즉 이 문제는 일반화(generalization) 문제가 아니라 **최적화(optimization) 문제**다.

2-3. 사고실험: Constructed Solution — 존재성과 도달 가능성은 다르다

저자들은 이 결론을 뒷받침하기 위해 사고실험을 하나 제시한다. 얇은 네트워크(예: 10층)가 이미 잘 학습되어 있다고 하자. 여기에 층을 더 얹어 20층으로 만들 때, "손해를 보지 않는" 구성법이 이론적으로 항상 존재한다.

1. 앞의 10층은 학습된 얇은 네트워크를 그대로 복사
2. 새로 추가한 10층은 모두 identity mapping(입력을 그대로 출력)으로 설정

이렇게 구성하면 20층 네트워크는 10층 네트워크와 완전히 동일하게 동작하므로, 학습 오차가 10층보다 나쁠 이유가 없다.

3. 핵심 아이디어: Residual Learning

3-1. 재정식화: $H(x) \rightarrow F(x) + x$

몇 개의 층이 근사해야 할 목표 사상을 $H(x)$ 라 하자. 여러 개의 비선형 레이어(conv + ReLU 등)를 쌓아서 "입력을 그대로 출력하라"는 identity mapping을 정확히 근사하는 것은, 목표 함수 $H(x)$ 자체를 처음부터 통째로 학습해야 하기 때문에 직관과 달리 최적화 관점에서 까다로운 문제다.

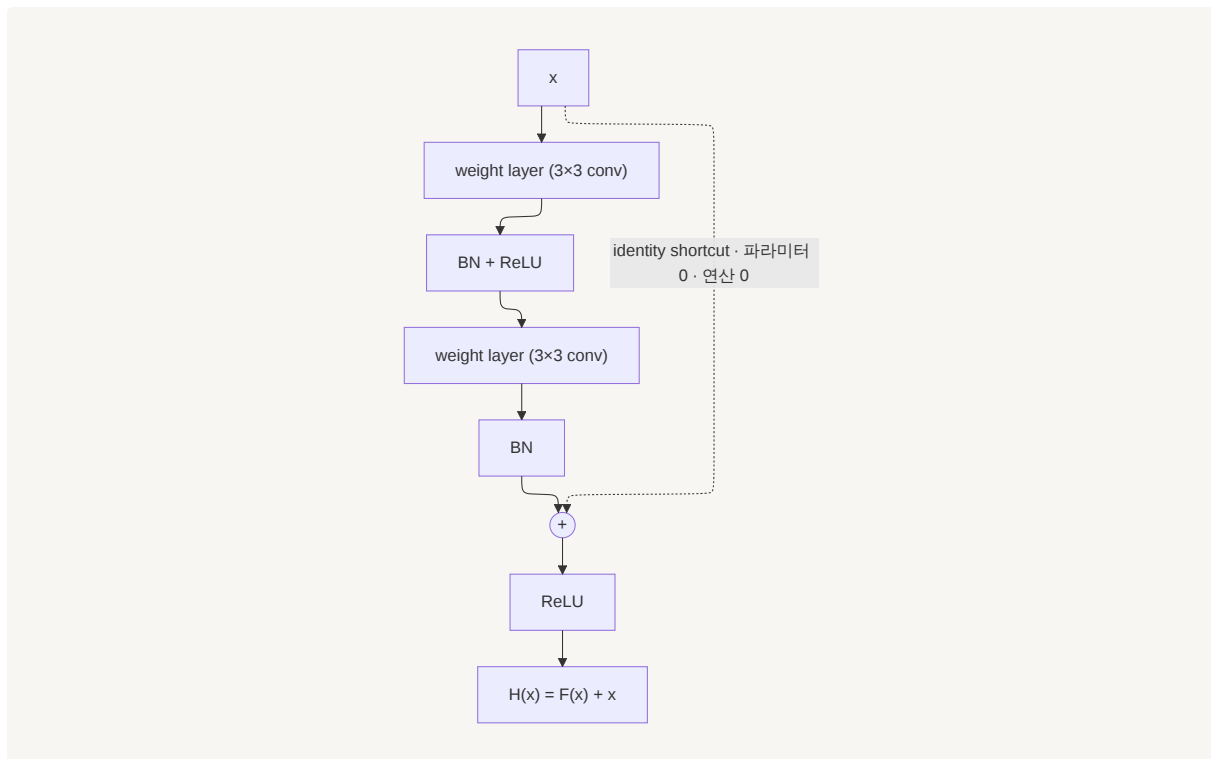
저자들은 이 층들이 $H(x)$ 를 직접 근사하는 대신, **잔차 함수**를 근사하도록 명시적으로 재구성한다.

$$F(x) := H(x) - x \quad \rightarrow \quad H(x) = F(x) + x$$

빌딩 블록은 다음과 같이 정의된다.

$$y = F(x, \{W_i\}) + x \quad \dots \text{식 (1)}$$

x , y 는 각각 이 레이어 묶음의 입력·출력이고, $F(x, \{W_i\})$ 가 학습 대상인 잔차 함수다. 2층 블록의 경우 $F = W_2 \cdot \sigma(W_1 x)$ (σ = ReLU, 편향 생략). shortcut connection은 identity mapping을 수행하고, 그 결과를 $F(x)$ 에 원소별(element-wise)로 더한 뒤 **덧셈 이후에** 두 번째 비선형성(ReLU)을 적용한다.



3-2. 왜 더 쉬운가 — Preconditioning 가설과 그 한계

두 형태($H(x)$ 직접 학습 vs $F(x)$ 학습) 모두 원하는 함수를 점근적으로 근사할 수 있다는 것은 이론적으로 동등하다. 하지만 **학습의 난이도가 다르다**는 것이 핵심 주장이다.

- **극단적인 경우:** identity mapping이 최적이라면, solver는 여러 비선형 층으로 identity를 힘들게 맞추는 대신 그저 **가중치를 0에 가깝게** 만들기만 하면 된다($F(x) \rightarrow 0$ 이면 출력은 자동으로 x). 훨씬 쉬운 문제다.
- **현실적인 경우:** identity mapping이 정확히 최적일 가능성은 낮다. 그러나 최적 함수가 영 사상(zero mapping)보다 identity mapping에 더 가깝다면, identity를 기준으로 한 작은 섭동(perturbation)을 찾는 편이 함수를 처음부터 새로 배우는 것보다 쉽다. 저자들은 이를 **preconditioning**이라 부른다.

이 가설의 실증적 근거가 Figure 7이다. CIFAR-10에서 각 3×3 conv layer 출력(BN 이후, 비선형성 이전)의 표준편차를 재보면, ResNet의 응답이 plain보다 전반적으로 작다. 또한 ResNet-20 $\rightarrow$ 56 $\rightarrow$ 110으로 깊어질수록 개별 층의 응답 크기가 더 작아지는 경향도 관찰된다. 즉 층이 많아질수록 각 층이 신호를 덜 건드린다는 뜻이다.

3-3. 놓치기 쉬운 디테일 — 덧셈 이후의 ReLU

shortcut 경로 자체는 파라미터도 연산도 없는 순수한 identity라 신호가 그대로 통과하지만, 두 블록이 이어지는 지점마다 그렇게 통과한 신호가 다음 블록으로 들어가기 전에 **ReLU를 한 번 더** 거친다(3-1의 식에서 $\sigma(y)$ 부분). 즉 "shortcut은 완전히 깨끗하다"는 설명은 블록 *내부*에는 맞지만, 블록과 블록 *사이*의 전체 forward path에는 완전히 들어맞지 않는다.

논문은 이 지점을 특별히 문제 삼지 않고 넘어가지만, 저자들 스스로 후속작(*Identity Mappings in Deep Residual Networks*, 2016)에서 BN $\rightarrow$ ReLU $\rightarrow$ conv 순서의 "pre-activation" 구조로 이 부분을 다시 설계한다 — 즉 이 시점의 설계는 완성형이 아니라 곧 갈아엎게 되는 1차 버전이었던 셈이다(11장에서 다시 다룸).

3-4. Identity Shortcut의 결정적 이점

identity shortcut은 **추가 파라미터도, 추가 연산 복잡도도 거의 없다**(원소별 덧셈 1회). 이것은 단순한 효율성 이슈를 넘어 **실험 설계상 핵심**이다 — plain 네트워크와 residual 네트워크를 파라미터 수·깊이·너비·연산량이 완전히 동일한 조건에서 비교할 수 있게 해주기 때문이다. 개선이 "파라미터가 늘어서"라는 반론을 구조적으로 차단하는 장치다.

3-5. 차원이 달라질 때 — Option A / B / C

x 와 F 의 차원이 다르면(채널 수가 바뀔 때) 선형 projection Ws 를 쓴다.

$$y = F(x, \{W_i\}) + W_s \cdot x \quad \dots \text{식 (2)}$$

옵션	방식	추가 파라미터	ResNet-34 top-1 (Table 3)
A	차원 증가분을 zero-padding, 모든 shortcut이 파라미터 없음	없음	25.03%
B (채택)	차원이 늘어나는 지점만 1x1 conv projection, 나머지는 identity	소량	24.52%
C	모든 shortcut을 projection	다수(13개)	24.19%

세 옵션 모두 plain보다 확연히 낮고, 셋 사이의 차이는 작다. 저자들은 **projection shortcut이 degradation 문제 해결에 본질적이지 않다**고 결론짓고, 메모리·연산·모델 크기를 줄이기 위해 B를 표준으로 채택한다. (오늘날 `torchvision.models.resnet`의 `downsample` 모듈이 바로 이 옵션 B이다.)

3-6. Highway Networks와의 차이

동시대 연구인 Highway Networks도 shortcut을 쓰지만 결정적으로 다르다.

	Highway Network	ResNet
shortcut 형태	데이터 의존적 게이팅 함수(파라미터 있음)	파라미터 없는 identity
게이트가 닫히면	non-residual 함수가 됨	닫히는 경우 자체가 없음 — 정보가 항상 통과
100층 이상 성능	정확도 항상 사레 없음	110층·1202층까지 학습 성공

여기에 더해, VLAD/Fisher Vector(잔차 벡터로 이미지를 인코딩하는 전통적 표현 기법)나 Multigrid(PDE를 여러 스케일의 잔차 하위 문제로 재구성하는 수치해석 기법) 같은 선례들도 언급되는데, 이는 "좋은 재구성(reformulation)이 최적화를 단순화한다"는 통찰이 이 논문만의 새로운 것이 아니라 이전부터 있었던 아이디어를 딥러닝에 성공적으로 적용한 것임을 보여준다.

4. 네트워크 아키텍처

4-1. Plain Network 설계 규칙

VGG net의 설계 철학을 따르되 두 가지 단순한 규칙을 적용한다.

1. 같은 크기의 출력 feature map을 만드는 레이어는 필터 수가 같다.
2. feature map 크기가 절반이 되면(다운샘플링), 레이어당 시간 복잡도를 보존하기 위해 필터 수를 두 배로 늘린다.

다운샘플링은 pooling이 아니라 stride 2 컨볼루션으로 직접 수행하며, 마지막은 global average pooling + 1000-way FC + softmax로 끝난다. VGG의 거대한 FC(4096×2) 두 층이 사라진 것이 연산량 차이의 큰 부분을 차지한다. **34층 plain 네트워크가 3.6 GFLOPs로, VGG-19(19.6 GFLOPs)의 18% 수준밖에 안 된다** — "더 깊다 ≠ 더 무겁다"를 보여주는 대목.

4-2. 연산 예산은 어디에 쓰였나 — conv4_x 집중

깊이를 늘릴 때 저자들은 균등하게 늘리지 않는다. 아래 표에서 보듯 101층에서 152층으로 51개 층을 더 넣을 때, 그중 36개(70% 이상)가 conv4_x(14×14 해상도 구간)에 들어간다.

단계/해상도	18층	34층	50층	101층	152층
conv2_x (56×56)	×2	×3	×3	×3	×3
conv3_x (28×28)	×2	×4	×4	×4	×8
conv4_x (14×14)	×2	×6	×6	×23	×36
conv5_x (7×7)	×2	×3	×3	×3	×3

채널 수가 이미 커져 있어(512) 3×3 conv의 연산량 자체는 무겁지만, 공간 해상도는 상대적으로 작다 — **표현력 대비 연산 비용이 가장 효율적인 지점**이라 여기에 깊이를 몰아넣은 것으로 해석할 수 있다. 다만 이 배분이 왜 하필 14×14 지점이 최적인지에 대한 정량적 분석은 논문에 없고, 결과적으로 그렇게 설계했다는 서술에 가깝다는 점은 짚어둘 만하다.

4-3. Bottleneck 블록 (50층 이상, Figure 5)

50층 이상에서는 학습 시간 예산 때문에 블록 구조를 바꾼다. 2층(3×3, 3×3) basic block 대신 3층(**1×1 → 3×3 → 1×1**) bottleneck block을 쓴다. 앞의 1×1이 채널을 줄이고, 3×3이 낮은 차원에서 실제 연산을 담당하고, 뒤의 1×1이 채널을 다시 복

원한다 — 입출력은 고차원, 중간만 저차원으로 만들어 "병목"을 형성하며, 두 설계의 시간 복잡도가 비슷해지도록 맞춘 것이다.

💡 **bottleneck에서 identity shortcut이 특히 중요한 이유:** bottleneck의 shortcut은 고차원 양 끝단에 연결된다. 여기를 projection으로 바꾸면 시간 복잡도와 모델 크기가 **2배**로 된다. 즉 identity shortcut은 "있으면 좋은 것"이 아니라 bottleneck 설계가 경제적으로 성립하기 위한 전제 조건이다.

4-4. Table 1 — 깊이별 구성과 연산량

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	공통: 7×7, 64, stride 2				
—	56×56	공통: 3×3 max pool, stride 2				
conv2_x	56×56	[3×3,64]×2 ×2	[3×3,64]×2 ×3	bottleneck ×3	bottleneck ×3	bottleneck ×3
conv3_x	28×28	[3×3,128]×2 ×2	[3×3,128]×2 ×4	bottleneck ×4	bottleneck ×4	bottleneck ×8
conv4_x	14×14	[3×3,256]×2 ×2	[3×3,256]×2 ×6	bottleneck ×6	bottleneck ×23	bottleneck ×36
conv5_x	7×7	[3×3,512]×2 ×2	[3×3,512]×2 ×3	bottleneck ×3	bottleneck ×3	bottleneck ×3
—	1×1	공통: average pool, 1000-d fc, softmax				
FLOPs		1.8G	3.6G	3.8G	7.6G	11.3G

가장 중요한 대비: **ResNet-152(11.3G)**가 **VGG-16(15.3G)/VGG-19(19.6G)**보다 여전히 가볍다. 8배 깊었는데 더 싸다.

5. 구현 세부사항 (Implementation)

항목	ImageNet	CIFAR-10
스케일 증강 / crop	짧은 변 [256,480] 랜덤 리샘플 → 224×224 랜덤 crop, 좌우반전	각 변 4px 패딩 후 32×32 랜덤 crop, 좌우반전
전처리	픽셀별 평균 차감 + 표준 색상 증강	픽셀별 평균 차감
정규화	매 conv 직후·활성화 이전에 Batch Normalization	동일
초기화	He 초기화, scratch 학습	동일
Optimizer	SGD, mini-batch 256	SGD, mini-batch 128(GPU 2장)
학습률	0.1 시작, 오차 정체 시 ±10	0.1 시작, 32k-48k iter에서 ±10
총 반복	최대 60×10 ⁴ iteration	64k iteration(45k/5k train-val split 으로 결정)
weight decay / momentum	0.0001 / 0.9	동일
Dropout	사용 안 함	사용 안 함
테스트	비교용 10-crop / 최종 성능은 fully-convolutional + multi-scale {224,256,384,480,640}	원본 32×32 단일 뷰

6. 실험 결과

6-1. ImageNet — Plain vs ResNet (Table 2, Figure 4)

동일 파라미터 수 조건에서의 top-1 오류율(10-crop, validation).

	plain	ResNet
18층	27.94	27.88
34층	28.54 (18층보다 나쁨 = degradation)	25.03 (18층보다 좋아짐)

읽어야 할 세 가지 관찰:

1. **역전이 일어난다.** plain은 34층이 18층보다 나쁘지만, ResNet은 34층이 18층보다 2.85%p 좋다. degradation이 이 설정에서는 잘 해결됐고, 깊이에서 실제로 이득을 얻는다.
2. **34층 ResNet은 학습 오차 자체가 낮다.** 즉 개선의 출처가 정규화 효과가 아니라 최적화 성공이다.
3. **18층에서는 plain과 ResNet 정확도가 비슷하다**(둘 다 "충분히 얕음"). 다만 ResNet이 훨씬 빠르게 수렴한다 — "not overly deep"한 경우에도 최적화 자체를 도와준다는 뜻.

저자들이 명시적으로 부정한 설명: 이 최적화 실패는 gradient vanishing 때문이 **아니다**(2-2절에서 확인). 그렇다면 진짜 원인은? 저자들은 "deep plain net의 수렴 속도가 지수적으로 느릴 것"이라 추측할 뿐, 원인 규명은 향후 연구로 남긴다.

6-2. Identity vs Projection Shortcut (Table 3)

A(zero-padding) < B(부분 projection) < C(전체 projection) 순으로 근소하게 좋아지지만 차이는 작다 → projection은 필수 아니며, 이후 실험은 메모리 효율을 위해 B 중심으로 진행한다.

6-3. 깊이 확장과 SOTA 비교

Table 3 — 10-crop 프로토콜

모델	top-1 err (%)	top-5 err (%)
VGG-16(저자 재현)	28.07	9.33
GoogLeNet	–	9.15
plain-34	28.54	10.02
ResNet-34 (A/B/C)	25.03 / 24.52 / 24.19	7.76 / 7.46 / 7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 4 — 단일 모델, dense(fully-convolutional) + multi-scale 프로토콜

모델	top-1 err (%)	top-5 err (%)
PReLU-net	21.59	5.71
BN-inception	21.99	5.81
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 5 — 앙상블, test set: ResNet-152 단일 모델의 4.49%는 이전 연구들의 앙상블 결과를 모두 능가한다. 여기에 깊이가 다른 6개 모델을 앙상블해 ****top-5 오류 3.57%****를 기록, ILSVRC 2015 분류 1위.

6-4. CIFAR-10 — 극단적 깊이 탐색 (Table 6)

목적은 SOTA 경신이 아니라 극단적으로 깊은 네트워크의 거동 관찰이다. 32×32 입력, 6n+2개 가중치 레이어, n = {3,5,7,9} → 20/32/44/56층. shortcut은 전부 옵션 A라서 plain과 깊이·너비·파라미터 수가 정확히 동일하다.

모델	파라미터	error (%)
Maxout / NIN / DSN	–	9.38 / 8.81 / 8.22
FitNet(19층)	2.5M	8.39
Highway(19층)	2.3M	7.54 (7.72±0.16)
ResNet-20	0.27M	8.75
ResNet-32	0.46M	7.51
ResNet-44	0.66M	7.17
ResNet-56	0.85M	6.97

모델	파라미터	error (%)
ResNet-110	1.7M	6.43 (6.61±0.16)
ResNet-1202	19.4M	7.93

- ResNet-110은 FitNet-Highway보다 파라미터가 적으면서 더 좋다. 깊고 얇은 구조 자체가 정규화 역할을 한다.
- **1202층도 최적화에는 성공한다** — 학습 오차 0.1% 미만. residual 학습이 1000층 규모에서도 무너지지 않는다는 것을 보여준다.
- 그런데 **테스트 성능은 110층보다 나쁘다**(7.93 vs 6.43). 저자들은 이를 오버피팅으로 해석하는데, 19.4M 파라미터는 CIFAR-10 규모에 비해 과하다는 것. 다만 이를 직접 검증하는 추가 실험(예: 정규화 강화)은 하지 않고 향후 과제로 남긴다.

놓치기 쉬운 실무적 디테일: ResNet-110은 초기 학습률 0.1로는 수렴을 잘 시작하지 못했다. 그래서 0.01로 워밍업해서 학습 오차가 80% 아래로 떨어질 때까지(약 400 iteration) 돌린 뒤 0.1로 되돌리는 트릭을 썼다. residual 구조가 최적화를 무조건 쉽게 만드는 것은 아니며, 깊이가 극단으로 가면 여전히 학습률 스케줄 개입이 필요하다는 뜻 — 오늘날 표준이 된 **learning rate warmup**의 초기 사례로 볼 수 있다.

6-5. Layer Response 분석 (Figure 7)

3-2절에서 다룬 내용과 같은 실험으로, ResNet의 층 응답이 plain보다 작고 깊을수록 더 작아진다는 것을 재확인. preconditioning 가설을 뒷받침하는 정황 증거로 기능한다(다만 인과관계까지 입증하는 것은 아니라는 점은 3-2절 참고).

6-6. 전이 성능 — Detection & Localization

Plain vs Residual 분류 비교는 핵심 증거이긴 하지만 "같은 저자가 같은 조건에서 만든 두 모델"의 비교라는 한계가 있다. 오히려 검출 파이프라인(Faster R-CNN)과 구현을 통째로 고정하고 backbone만 VGG-16 → ResNet-101로 교체한 아래 실험이 더 통제된 비교다.

데이터셋	지표	VGG-16	ResNet-101
PASCAL VOC 07test	mAP	73.2	76.4
PASCAL VOC 12test	mAP	70.4	73.8
COCO val	mAP@.5	41.5	48.4
COCO val	mAP@[.5,.95]	21.2	27.2 (28% 상대 개선)

흥미로운 점은 절대 상승폭이 mAP@.5(+6.9)와 mAP@[.5,.95]에서 거의 비슷하다는 것이다. 후자는 IoU 기준이 훨씬 엄격해서 박스 위치 자체의 정밀도까지 요구하는 지표인데, 여기서도 비슷한 폭으로 개선됐다는 건 더 깊은 표현이 분류뿐 아니라 **공간적 정밀도(localization)까지** 함께 끌어올린다는 뜻이다. 이 결과는 "residual learning이 특정 벤치마크에 최적화된 트릭이 아니라 범용적으로 더 나은 표현을 만든다"는 주장을 가장 잘 뒷받침하는 증거로 볼 수 있다.

추가 개선 기법(box refinement, global context, multi-scale testing)을 누적 적용하면 COCO test-dev에서 mAP@.5 59.0%, mAP@[.5,.95] 37.4%까지 올라가 COCO detection 2015 1위를 차지했고, 이 모델을 기반으로 ImageNet Detection(mAP 62.1%, 2위 대비 8.5점 차), ImageNet Localization(top-5 오차 9.0%, 이전 최고 대비 상대 64% 개선)에서도 1위를 달성했다.

7. 논문의 기여 정리

1. **Degradation 문제를 명확히 규명:** 깊은 네트워크의 성능 저하가 overfitting이 아니라 최적화의 어려움임을 여러 대안 가설을 소거하며 실험적으로 입증
2. **Residual learning 재구성 제안:** $H(x)$ 대신 $F(x)=H(x)-x$ 를 학습하는 단순한 재구성으로 문제 완화
3. **Parameter-free identity shortcut:** 추가 파라미터·연산량 없이 구현 가능 → 공정한 비교와 손쉬운 적용 모두 확보
4. **극단적 깊이에서의 실증:** 152층(ImageNet), 1202층(CIFAR-10)까지 학습 가능함을 보여줌
5. **일반화 가능성 입증:** 분류뿐 아니라 detection·localization·segmentation 등 다른 비전 과제에도 표현력이 잘 전이됨을 4개 대회 1위로 입증
6. **연산 경제성까지 설계에 포함:** 정확도만 좇지 않고 FLOPs를 VGG 이하로 유지하면서 깊이를 확보 → 이후 산업 채택 속도에 직접 영향을 줬을 실용적 기여

8. 강점

- **문제 재정의가 논리적이다.** "깊으면 왜 나빠지는가"를 표현력 문제가 아니라 최적화 문제로 규정하고, constructed solution 논증으로 이를 논리적으로 뒷받침한다.
- **대안 가설을 능동적으로 배제한다.** gradient vanishing이 아님을 BN·gradient norm 확인으로, 반복 부족이 아님을 3배 학습으로, 오버피팅이 아님을 학습 오차로 각각 반박하는 방식으로 자기 주장의 신뢰도를 높인다.
- **해법이 단순하다.** 덧셈 하나. 새로운 solver도, 라이브러리 수정도, 특별한 초기화도 필요 없다.
- **실험 설계가 공정하다.** identity shortcut이 파라미터를 추가하지 않는다는 성질을 이용해 plain과 residual을 완전히 동일한 조건에서 비교한다.
- **연산 경제성을 설계에 포함시켰다.** 정확도만 쫓지 않고 FLOPs를 VGG 이하로 유지하면서 깊이를 확보했다.

9. 한계 및 비판적 검토

9-1. "왜 되는가"에 대한 완결된 이론이 없다. 저자들은 plain 네트워크가 왜 최적화에 실패하는지 끝내 설명하지 못한다.

gradient vanishing이 아님은 배제했지만, 대안으로 제시한 "수렴 속도가 지속적으로 느릴 것"이라는 설명은 뒷받침 증거 없는 추측이며, 원문 그대로 "향후 연구 과제"로 남겨진다. residual 학습이 왜 도움이 되는지(preconditioning)에 대한 설명도 직관적 서술 + Figure 7의 정황 증거에 머물며, 상관관계와 인과관계가 명확히 구분되지 않는다. 각주에는 다층 비선형 층이 복잡한 함수를 점근적으로 근사할 수 있다는 전제 자체가 "여전히 열린 질문"이라는 정직한 고백도 있다.

9-2. Option C를 버린 근거가 약하다. Table 3에서 가장 좋은 것은 C(24.19%)인데, 저자들은 B(24.52%)를 채택하고 C의 우위를 "13개의 projection shortcut이 추가한 파라미터 덕분"이라고 사후 해석한다. 이 해석을 검증하는 통제 실험(예: 파라미터 수를 맞춘 비교)은 없다. 결론 자체(비용 대비 이득이 작다)는 합리적이지만, 논증이라기보다 판단에 가깝다.

9-3. ResNet-1202의 실패 원인이 검증되지 않았다. 1202층이 110층보다 나쁜 것을 오버피팅으로 단정하지만 이를 확인할 실험은 하지 않았다. 저자들 스스로 "더 강한 정규화(maxout/dropout)와 결합하면 개선될 수 있고, 이는 향후 연구"라고 적는다. 다만 이는 "최적화의 어려움"이라는 논문의 초점을 흐리지 않기 위한 의도적 선택이기도 하므로, 변호 가능한 선택이지만 결론의 강도는 그만큼 낮춰 읽어야 한다.

9-4. 블록 설계에 대한 ablation이 부족하다. BN·ReLU·덧셈의 순서(3-3절의 post-activation 이슈)를 왜 그렇게 배치했는지에 대한 실험적 근거가 없다. bottleneck 도입 근거도 "감당 가능한 학습 시간"이라는 실용적 이유이지 원리적 정당화는 아니다. 그 결과 **ResNet-34 vs ResNet-50 비교가 완전히 공정하지는 않다** — 깊이(34 → 50)와 블록 구조(basic → bottleneck)가 동시에 바뀌기 때문에, 성능 향상을 어느 쪽에 귀속시킬지 이 논문만으로는 알 수 없다.

9-5. 비교 대상의 프로토콜이 섞여 있다. Table 3의 VGG-16 수치는 원저자 보고값이 아니라 저자들의 재현 결과("based on our test")다. 또한 6-3절에서 짚었듯 Table 3(10-crop)과 Table 4(dense+multi-scale)의 프로토콜이 달라, 같은 모델의 수치를 옮겨 적을 때 혼동하기 쉽다.

9-6. "연산 복잡도 증가 없음"은 절반만 참이다. identity shortcut이 FLOPs와 파라미터를 늘리지 않는 것은 사실이지만, 실제 학습에서는 shortcut을 위해 입력 텐서를 메모리에 살려둬야 하고 원소별 덧셈은 메모리 대역폭을 소모한다. FLOPs로 측정되지 않는 비용이 존재하며, 이는 실측 학습 속도와 GPU 메모리 사용량에 영향을 준다.

9-7. 깊이 배분(conv4_x 집중, 4-2절)의 근거가 사후적이다. 왜 하필 14×14 지점이 최적인지에 대한 정량적 분석 없이, 결과적으로 그렇게 설계했다는 서술에 가깝다.

9-8. 깊이와 너비가 분리되지 않았다. 논문 전체가 "깊이가 핵심"이라는 서사로 구성되어 있지만, 깊이를 늘리는 것과 너비(채널 수)를 늘리는 것을 비교하는 실험은 없다. 이 서사는 아래 11장의 후속 연구에서 실제로 도전받는다.

9-9. "shortcut이 완전히 clean하다"는 인상을 준다. 3-3절에서 짚었듯, 블록 내부의 identity는 확실히 clean하지만, 덧셈 이후 ReLU가 블록 사이 신호를 계속 비트는 문제는 본문에서 언급조차 되지 않는다. 저자들 스스로 후속작에서 뒤집는 설계 선택이라는 점에서, 당시 시점에는 완전히 인지되지 않았던 한계로 보인다.

10. 이후 연구가 채운 공백

후속 연구	ResNet의 어떤 공백을 다뤘나
Identity Mappings in Deep Residual Networks (He et al., ECCV 2016)	9-9의 post-activation 문제. BN → ReLU → conv 순서("pre-activation")로 바꿔 shortcut을 블록 간에도 완전히 clean하게 만들고, CIFAR에서 1001층 학습에 성공. 저자들 본인의 직접적인 후속편

후속 연구	ResNet의 어떤 공백을 다뤘나
Wide Residual Networks (Zagoruyko & Komodakis, 2016)	9-8의 깊이-너비 문제. 알지만 넓은 ResNet이 아주 깊은 ResNet보다 빠르고 정확할 수 있음을 보여, "깊이가 전부"라는 서사에 반례 제시
Squeeze-and-Excitation Networks (2018)	4-2의 "연산 예산을 어디에 쓸까" 논의를 확장. 깊이-너비가 아니라 채널 간 중요도(attention)를 재배분하는 축을 추가해 같은 연산 예산에서 더 큰 개선
Residual Networks Behave Like Ensembles of Relatively Shallow Networks (Veit et al., 2016)	9-1의 설명 공백. ResNet을 지수적으로 많은 경로의 암묵적 앙상블로 해석 (unraveled view). 대부분의 gradient가 짧은 경로에서 온다는 분석 제시
Visualizing the Loss Landscape of Neural Nets (Li et al., 2018)	9-1의 설명 공백. skip connection이 손실 표면을 매끄럽게(smooth) 만들어 최적화를 쉽게 한다는 시각적·정량적 증거 제시
DenseNet(2017), ResNeXt(2017)	연결 구조와 블록 설계의 확장 — dense connectivity, grouped convolution
Transformer 계열 전반	residual connection + normalization이 CNN을 넘어 거의 모든 딥러닝 아키텍처의 기본 구성요소가 됨. Transformer의 "residual stream" 개념이 직접적 후손

11. 구현 관점 체크포인트 (실습·재현 시 참고)

- shortcut의 다운샘플링도 stride 2를 적용해야 한다. 본 경로와 크기가 안 맞으면 덧셈이 불가능하다.
- `torchvision.models.resnet`의 `downsample` 모듈이 곧 옵션 B(1×1 conv + BN)다. A/B/C 중 어느 것을 쓰고 있는지 인지하고 쓰는 게 좋다.
- ReLU는 원 논문 기준으로는 **덧셈 이후**에 적용한다. pre-activation 변형(후속 논문)과 헷갈리지 않도록 주의.
- 아주 깊은 모델을 학습할 때는 6-4절의 110층 사례처럼 **learning rate warmup**을 고려한다.
- Faster R-CNN으로 전이할 때 저자들은 사전학습 후 **BN 통계를 고정**하고 fine-tuning했다(메모리 절감 목적). 배치 크기가 작은 detection 학습에서 BN이 불안정해지는 문제와 직결되는 실무 포인트.

12. 느낀 점(팀원 별)

김석우

이번엔 "잔차를 학습한다"는 아이디어 자체보다, **용의자를 하나씩 지우는 검증 방식**이 더 눈에 들어왔다. 대학원 연구 분야로 생각 중인 'FEM 결과를 surrogate model로 근사할 때'도 "모델이 안 맞는 이유"를 데이터 부족·구조 부족·최적화 실패 중 어디로 돌릴지 헷갈리는 경우가 많은데, 이 논문처럼 각 원인을 지울 수 있는 독립적인 진단 실험(예: 학습 곡선만으로 overfitting과 최적화 실패를 구분하는 것)을 미리 설계해두는 습관을 여기서 가져가고 싶다.

오은하

가장 인상 깊었던 것은 $F(x) + x$ 라는 단순한 재구성이 딥러닝을 매우 깊은 층까지 학습할 수 있게 만들었다는 점이다. 특히 논문에서 오버피팅, 기울기 소실, 학습 부족이라는 원인들을 하나씩 실험으로 반박해 나가는 소거법적 접근이 설득력 있게 다가왔다. 앞으로 이렇게 어떤 현상의 원인을 여러 가설로 나눠 검증하는 문제 해결 방식과, 복잡한 모듈을 새로 설계하기보다 최적화 자체를 쉽게 만드는 단순한 재구성을 먼저 고민하는 태도를 배우고 싶다는 생각이 들었다.

이신영

단순히 층을 많이 쌓는 것이 항상 좋은 것은 아니라는 점이 가장 인상적이었다. 지금까지는 네트워크가 깊어질수록 더 복잡한 특징을 학습할 수 있기 때문에 성능도 자연스럽게 좋아질 것이라고 생각했는데, ResNet 논문을 읽으면서 모델의 깊이와 성능은 별개의 문제라는 것을 알게 되었다. 이후 Transformer나 다른 모델을 공부할 때도 단순히 "어떤 구조를 추가해서 성능을 높였는가"만 보는 것이 아니라, 왜 기존 구조로는 학습하기 어려웠고 새로운 구조가 그 문제를 어떻게 해결했는지를 중심으로 논문을 읽어봐야겠다고 느꼈다.

정지민

처음엔 shortcut이 그저 gradient가 잘 흐르도록 돕는 장치인 줄 알았는데, 읽다 보니 파라미터가 전혀 늘어나지 않는다는 점이 더 크게 다가왔다. 덕분에 plain 네트워크와 ResNet을 완전히 동일한 조건에 두고 비교할 수 있었고, 성능 향상이 오직 '깊이 (depth)' 덕분이라는 점을 명확히 증명해 냈다. 파라미터가 조금이라도 늘어나는 구조였다면 이런 주장을 펼치지 못했을 테니, 이 설계는 우연이 아니라 철저히 계산된 선택이었던 셈이다. 좋은 아이디어를 떠올리는 것 못지않게, 그 아이디어가 맞다는 것을 완벽히 입증할 수 있도록 '실험적 판'을 짜는 일이 얼마나 어려운지 새삼 느끼게 된다.